Crux—A New Fast, Flexible and Decentralized Consensus Algorithm with High Fault Tolerance Rate

Pengfei Li[1,2](✉), Jingtian Peng[1], Long Yang[2], Qian Zheng[3], and Gang Pan[2]

[1] LD Research, Shanghai, China
pfli0220@gmail.com, jingtian.peng@ldres.org
[2] Zhejiang University, Hangzhou, China
{yanglong,gpan}@zju.edu.cn
[3] Nanyang Technological University, Singapore, Singapore
csqianzheng@gmail.com

Abstract. This paper presents Crux, a new permissionless blockchain consensus algorithm that achieves higher fault tolerance rate with more flexibility than existing blockchains such as Bitcoin, Ethereum and EOS. Crux utilize a DPoS-XPaxos pipelined algorithm to achieve effective and efficient consensus. Those who hold tokens in Crux elect $2f + 1$ block producers called validators through a continuous approval voting system. The elected validators are scheduled in an order and produce blocks in turns agreed by all of the validators. XPaxos, guarantees $\frac{f}{2f+1}$ fault tolerance rate, is added to traditional DPoS to confirm blocks. Once $f + 1$ validators have signed a block, it is deemed irreversible. Analysis shows Crux provides higher securities, better flexibility, higher TPS (transaction per second) with little cost of centralization compared with existing blockchain consensus algorithms.

Keywords: Blockchain · Consensus algorithm · DPoS · XPaxos Fault tolerance

1 Introduction

Traditional financial system is inefficient and expensive in global transaction: every participator (or a centre organization) should take part in the maintenance of its own account. Besides, the account in database cannot be fully trusted by all the participators in the multi-stakeholder transactions. Blockchain, a decentralized, tamper-resistant, traceable and distributed ledger, is a potential way to overcome above difficulties and improve traditional financial ecosystem.

Blockchain is originated in the technology of Bitcoin [22], and due to the success of Bitcoin crypto-currency, a lot of blockchain systems have been proposed in recent years, such as Ethereum [4], Hyperledger Fabric [5], BigchainDB [21]

The intellectual property right of this paper belongs to LD Research.

© Springer Nature Switzerland AG 2018
M. Qiu (Ed.): SmartBlock 2018, LNCS 11373, pp. 66–76, 2018.
https://doi.org/10.1007/978-3-030-05764-0_7

and et al. Comparing to traditional financial ecosystem, one of the most significant advantages in blockchain system is that it is unnecessary to ensure all the participators are trusted by each other and it is designed to achieve the Byzantine fault tolerance. Thus, consensus algorithm, the process of agreeing on a deterministic order of transactions and filtering invalid transactions is one of the core issues in blockchain. For example, Proof-of-Work (PoW) [22], Proof-of-Stake (PoS) [12], Delegated Proof-of-Stake (DPoS) [18] and Practical Byzantine Fault Tolerance (PBFT) [8] are typical approaches proposed to achieve agreement on the transactions.

Existing consensus protocols suffer from different shortcomings. For example, even though PoW guarantees a fault tolerance rate of 50%, it is computationally expensive with low utilization, and the forks produced by it makes the block chaim system less stable and reliable. Forks are also the main problem in PoS. People staking their coin can vote for both forks of the blockchain, and can even mine effortlessly in secret. Thus, PoS is not able to achieve consensus, unfortunately. In DPoS, only 21 nodes are elected as block producers and a block generation needs confirmations from at least 15 producers, *i.e.*, the fault tolerance rate is less than $\frac{1}{3}$. Similarly, PBFT can only achieve a fault tolerance rate of less than $\frac{1}{3}$—it is not high enough.

To address above problems, in this paper we propose Crux, a new consensus protocol that contains two parts: (1) a small subset of nodes (active delegates called "validators" in this paper) is elected firstly to produce blocks, (2) then confirm blocks by an XFT [20] model. Crux provides a fault tolerance rate of $\frac{f}{2f+1}$—larger than that of PBFT and DPoS. Moreover, it is unnecessary to generate forks and is more flexible and efficient.

For the delegates election step in Crux, we adopt the method introduced in Delegated Proof of Stake (DPoS) [1], the users (or nodes) who hold tokens in Crux elect $2f+1$ validators, *i.e.*, the block producers. Only the top $2f+1$ witnesses (who have collected the most votes) gain the right to turn to the following validate transactions part. In Crux, the voting power that the token holder has, otherwise known as voting weight, is determined by how many of the base token the account is holding. The roles of validators in Crux revolve around as follows: ensuring they are always up and running, collecting the transactions across the network into blocks, and signing and broadcasting those blocks, validating the transactions.

For the confirmation part in Crux, we use XPaxos protocol which is proposed in XFT model. This part makes the proposed Crux be an efficient resilient blockchain system that tolerates both Byzantine faults and crash faults (as well as network faults). Traditional consensus protocols such as Fault-tolerance based algorithms such as Practical byzantine fault tolerance [8] (PBFT), Zyzzyva [15] and Raft [23] give extraordinary power to the adversary, which results in there is no practical solution to effectively tolerate Byzantine faults. The XPaxos in Crux put a few restrictions on the power to the adversary and achieves higher fault tolerance rate. Besides, the XPaxos does not consume extra resources and preserve all reliability guarantees of asynchronous crash fault-tolerance

(CFT) [20], *e.g.*, Paxos [17]. It is an efficient and effective way to provide correct service as long as a majority of validators are correct and communicate synchronously.

Our contributions in this study are mainly the superior advantages of Crux, which can be summarized as follows:

- Crux provides a $\frac{f}{2f+1}$ fault tolerance rate. As long as more than f validators are correct and synchronous, it is guaranteed that blocks could be generated and confirmed correctly and efficiently.
- Crux will not generate forks. Upon a block is appended to the chain, it is stabilized and irreversible.
- Crux is flexible. The number of validators and the expiration time of validators' generating blocks could be dynamically adjusted.

This paper is organized as follows. Section 2 details our Crux. Section 3 reports the analysis of Crux. Section 4 reviews the related works. Finally, Sect. 5 concludes the paper and points to future work directions.

2 Crux Details

Existing blockchain consensus algorithms face many problems: Low fault tolerance rate (PBFT, DPoS); Huge computation resource consumption (PoW); Forks (PoW, PoS, DPos). These problems will bring insecurity or resource waste. To address these problems, we propose a scheme that a small subset of nodes is elected firstly to produce blocks. Then XPaxos, a novel state-machine replication (SMR) protocol designed in the XFT model [20], is used to confirm blocks.

2.1 Notations

We use N to denote the total number of all nodes in the Crux. Since any node is able to join or leave the Crux at any time arbitrarily, N is a dynamically changing number. These N nodes elect a group of validators, *i.e.*, block producers at intervals. We use *epoch* and Δep to denote the succession of node configurations from the beginning of one election to the next one and the expiration time of an epoch, respectively. In each epoch, $n = 2f + 1$ nodes are elected as validators.

Validators may suffer from the following kinds of faults: (1) Byzantine faults, where a validator act arbitrarily, *i.e.*, it fails to deliver blocks, delay them or duplicate them, but cannot subvert the cryptographic techniques we use (public-key signatures [26], message authentication codes [27], message digests produced by cryptographic hashes [7,25]); (2) crash faults [20], where a validator may suffer from a system crash or network break so that it stops all computation and communication. A node that is neither Byzantine faulty nor crash faulty is called *correct*.

We assume these validators are connected by point-to-point network asynchronously. Thus, validators may suffer network faults, *i.e.*, two replicas cannot deliver and process message within a delay Γ. We give the following definition to help quantify the number of network faults [20].

Definition 1 (Partitioned validator). *A validtor v is partitioned if it is not in the largest network-connected component, in which every pair of validitors can deliver and process messages among each other within Γ.*

Figure 1 shows an example about partitioned validators. We say a validator v is synchronous if v is not partitioned.

Having above descriptions, the number of three kinds of validator faults at a given moment s is modeled by

- $t_c(s)$: the number of crash-faulty nodes.
- $t_b(s)$: the number of Byzantine-faulty nodes.
- $t_p(s)$: the number of correct, but partitioned nodes.

Definition 2 (Anarchy). *The group of validators is in anarchy at a given moment s iff $t_c(s) + t_b(s) + t_p(s) > f$.*

Definition 3 (XFT protocol). *Protocol P is an XFT protocol if P satisfies safety in all executions in which the system is never in anarchy.*

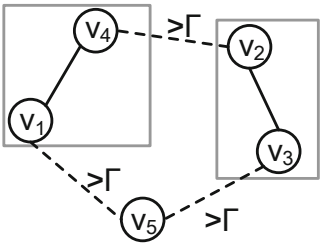

Fig. 1. An illustration of partitioned nodes: $\{v_1, v_4, v_5\}$ or $\{v_2, v_3, v_5\}$ are partitioned based on Definition 1 [20].

2.2 DPoS in Crux

Crux utilizes Delegated Proof of Stake (DPoS) [1], an existing decentralized consensus algorithm that is used in some popular blockchains such as EOS [2] and has been proven to be capable of meeting the requirements of applications on the blockchain, to elect validators on the beginning of each epoch. Under DPoS, those who hold tokens on Crux elect block producers through a continuous approval voting system. Anyone may choose to participate in block production and will be given an opportunity to produce blocks, provided they can persuade token holders to vote for them.

Under DPoS, the stakeholders can elect any number of nodes to be validators and generate blocks. Each account is allowed one vote per share per validator, a process known as approval voting. The top $n = 2f + 1$ nodes by total approval are selected. The number (n) of validators is defined such that at least 50% of

voting stakeholders believe there is sufficient decentralization. Thus, it may be changed: when stakeholders think the speed of blocks generation is too low, the number will decrease; when stakeholders want more decentralizations, they will demand increase n. This process is also executed by voting: when stakeholders expresses their desired number of validators, they must also vote for that many validators. Crux will choose the median number of the voting numbers of n.

Each time validators produce a block, they will be paid for their services. If a validator fails to produce a block, then they will not be paid, and may be voted out in the future. In addition, those nodes participating the election and voting for the validators that generate blocks successfully will be also paid. This ensures that stakeholders are ultimately in control because stakeholders will get paid if they act correctly (*i.e.*, to vote for the ones that are most probable to become validators) but lose the most when they does not operate smoothly. For now, the pay rates of the validators that generated blocks and their voters are set by the stakeholders via their elected delegates, same with BitShares [18].

2.3 XPaxos in Crux

Crux utilizes XPaxos, a state-machine replication protocol in the XFT model [20] to make validators generate and confirm blocks. In each epoch *ep*, XPaxos is orchestrated in a sequence of *views* [8] (*i.e.*, a succession of validator configurations.). It consists of two main components:

- A common-case protocol, which packs transactions into blocks, then replicates and totally orders them across the networks.
- A new view-change protocol, in which the information is transferred from one view to another. It occurs when faulty behaviors of invalid validators are observed by valid validators or the current view expires. It is performed in a decentralized, leaderless fashion.

Each view is assigned to a unique number which is incremented once view-change occurs. For a given view i, we use a mapping known to all validators to determine a synchronous group sg_i which consists one *primary* and f *followers*—they are jointly called active validators. The remaining f validators in a given view are called *passive* validators. We denote the digest of a message m (maybe a block or some other kinds of information) by $D(m)$, where $\langle m \rangle_{\sigma_v}$ denotes a message that contains both $D(m)$ signed by the private key of validator v and m. We assume all nodes have public keys of all other nodes.

Common Case. In the common case, new digitally signed transactions are broadcast to all validators and each validator keeps a transactions pool. The primary collects transactions from his transactions pool into a block and sign it. Then this block is sent to other t active validators to be confirmed.

To be more specific, upon a block b is generated from the primary (says s_0), s_0 (1) increments the block number bn and assign bn to b; (2) signs a message $prep = \langle \text{PREPARE}, D(b), bn, i \rangle_{\sigma_{s_0}}$ and *prepares* the block b, *i.e.*, logs $\langle b, prep \rangle$

into its prepare log $PrepareLog_{s_0}[bn]$, and (3) forwards $\langle b, prep\rangle$ to all other active validators (*i.e.*, the f followers).

Each follower s_j $(1 \leq j \leq f)$ verifies the validity of each transaction in the block b and the primary s_0's signature. It will also be checked that if the local block number to be equal to $bn - 1$. If all of these validity verifications are achieved, s_j (1) logs $\langle b, prep\rangle$ into its prepare log $PrepareLog_{s_j}[bn]$, (2) update its local block number to bn, (3) signs the digest of the block b, the block number bn and the view number i, and (4) sends $\langle \text{COMMIT}, D(b), bn, i\rangle_{\sigma_{s_j}}$ to all active validators (including the primary).

If an active validator s_k $(0 \leq k \leq f)$ receives f signed COMMIT messages—one from each follower, it (1) *commits* the block b, *i.e.*, logs *prep* and the f signed COMMIT message into its commit log $CommitLog_{s_k}[bn]$, and (2) *confirms* the block b, *i.e.*, broadcasts the authentication message $\langle$AUTHENTICATION, $b, bn, ep, \rangle_{\sigma_{s_k}}$ to all other nodes (including those non-validator nodes) and remove all transactions in b from its transaction pool. A node accepts the block b and appends it to the chain of Crux when it receives $f+1$ matching valid AUTHENTICATION messages. An AUTHENTICATION message is treated valid if (1) s_k is an elected validator in epoch ep, (2) the block number bn is larger than the local block number of a node and (3) every transaction in the block b is valid. Note that if a node cannot find the previous block of b based on b's previous block pointer, it will wait until b's previous block arrives, similar to Bitcoin [22].

The common-case message pattern of XPaxos in Crux is shown in Fig. 2.

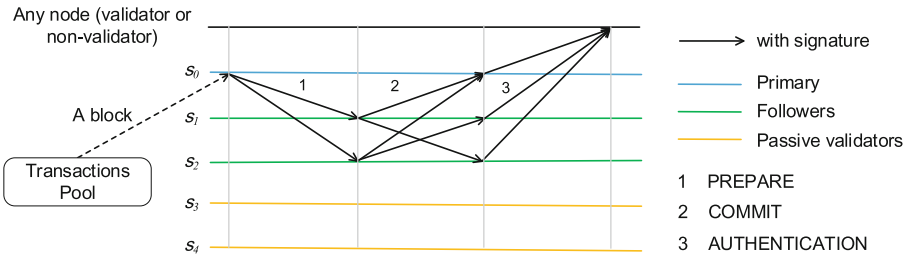

Fig. 2. Common-case message pattern of XPaxos in Crux when $f = 2$.

View-Change. Different from traditional view change techniques [9,10,14] where the view-change is led by a single node (usually the primary), in XPaxos view-change from view i to $i+1$, every active validator in view $i+1$ participate in the process.

We assign a expiration time Δv for each view. The objective is to make every validator have chances to be the primary. If view i expired, or sg_i, the synchronous group in view i does not make progress, *i.e.*, some faults are detected, a view-change occurs. Specifically, an active validator $s_k \in sg_i$ initiates a view change if (1) view i expires, or (2) s_k does not receive PREPARE message in Γ if

it is not the primary, or (3) s_k does not receive enough COMMIT or AUTHENTICATION messages in Γ, or (4) s_k receives a message from other active validator that does not conform to the protocol (*e.g.*, an invalid signature or transactions in the block is invalid), or (5) s_k does not complete a view change to view i in a timely manner, or (6) s_j receives a valid SUSPECT message for view i from another active validator in sg_i, s_k will stop participate in the current view upon a view change is initiated. Then it will send $\langle \text{SUSPECT}, i, s_k \rangle_{\sigma_{s_k}}$ to all other validators (including the passive ones).

Upon receiving a SUSPECT message from an active validator in view i, validator s_k stops processing messages of view i and sends $\langle$VIEW-CHANGE, $i+1, s_k, PrepareLog_{s_k}, CommitLog_{s_k} \rangle_{\sigma_{s_k}}$ to the $f+1$ active validators of sg_{i+1}. Note that $PrepareLog_{s_k}$ and $CommitLog_{s_k}$ might be empty if s_k is passive. Besides, the synchronous groups of all views are deterministic to all validators, *i.e.*, each synchronous group uniquely determines the primary and every validator knows the members of the synchronous group in a given view. Active validator s_k in view $i+1$ waits for 2Γ time to collect as many VIEW-CHANGE messages. If at least $f+1$ VIEW-CHANGE message are received from all, s_k inserts all these messages into set $VCSet_{s_k}^{i+1}$. Afterwards s_k sends $\langle$VC-FINAL, $i+1, s_k, VCSet_{s_j}^{i+1} \rangle_{s_k}$ to every active validator in view $i+1$. Active validators are able to exchange the received VIEW-CHANGE messages through this process.

To start the new process of generating blocks and delivering messages, every active validator $s_k \in sg_{i+1}$ must receive VC-FINAL messages from all active validators in sg_{i+1}. Afterwards, the new primary ps_{i+1} collects transactions from its transaction pool and generate a new block b. Then it processes the block as described in the common case, *i.e.*, it increments the block number, prepares b and forwards $\langle b, prep \rangle$ to all followers in the new view. Figure 3 shows the message pattern.

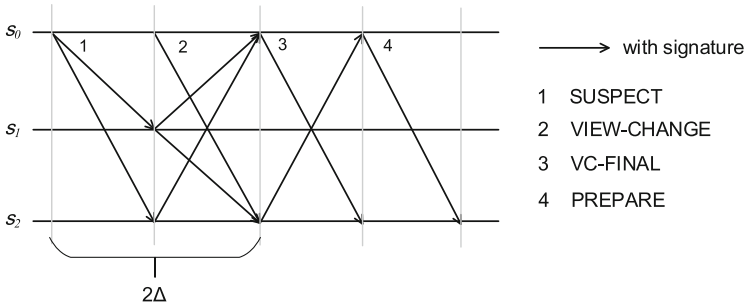

Fig. 3. XPaxos view change: the synchronous group is changed from $\{s_0, s_1\}$ to $\{s_0, s_2\}$.

3 Crux Analysis

3.1 Correctness of Crux

The correctness of Crux is guaranteed if the synchronous group contains only correct and synchronous validators. As long as the total number of Byzantine-faulty, crash-faulty and partitioned validators among $2f+1$ validators in an epoch is less than f, *i.e.*, $t_b(ep) + t_c(ep) + t_p(ep) < f$, this can be achieved. Note that, there exists at least $f+1$ correct and synchronous validators, which are able to compromise a synchronous group sg—that is what we demand. Recall that, outside anarchy, in a given view i, at least one correct and synchronous active validator exists. These validators will report the invalid behaviors of faulty/partitioned validators and invoke a view change. A new synchronous group will replace the current one. The replacement process will continue until a synchronous group containing correct and synchronous validators only (sg is chosen). XPaxos guarantees that, eventually, view change will complete with $f+1$ correct and synchronous active validators. It is also indicated from above descriptions that the fault tolerance rate of our Crux is $\frac{f}{2f+1}$.

3.2 Confirmation of Blocks

As we described in Sect. 2.3, the process of a block generation needs $f+1$ confirmations, one from each active validator in a synchronous group. As we assume at most f validators are incorrect or partitioned, in this case a block in the chain is confirmed by at least one correct and synchronous active validator so that it is valid. In addition, Crux will not generate forks. Therefore, upon a block is appended to the chain, it becomes stabilized and irreversible.

In addition, due to our Crux confirm blocks based on messages deliveries and does not need to calculate nonces satisfied to some conditions like PoW, the blocks confirmation speed or transactions per second (TPS) of a blockchain system employing our Crux is much higher than that of blockchain systems using PoW as their consensus algorithm (*e.g.*, Bitcoin [22] and Conflux [19]).

3.3 Flexibility of Crux

We provide two kinds of flexibility in Crux:

- The number of validators can be dynamically changed. In contrast, in DPoS, the number of delegated block producers is fixed to 21. By reducing the number, the COMMIT and AUTHENTICATION messages delivery time would be shortened so that the process of block confirmation can be finished in a shorter time period. This servers to be a kind of dynamics that controls the block generation speed. The function of adjusting the number of validators is similar to the difficulty adjustment in Bitcoin system [22].

- Δep and Δv—the epoch and view expiration time, respectively, can be changed. By decreasing Δep, more nodes have chances to become validators. On the contrary, if in a time period, elected validators are all correct and synchronous so that they can generate blocks efficiently and correctly. We can increase Γ to make trustable validators can generate blocks for longer time. Similarly, by changing Δv, we can adjust the speed of view change.

4 Related Work

In blockchain, how to reach consensus among the untrustworthy nodes is a transformation of the fault tolerant (FT) Problem, which was raised in [16]. In blockchain, there is no central node that ensures ledgers on distributed nodes are all the same. Some protocols are needed to ensure ledgers in different nodes are consistent.

The early BFT protocols are either too expensive, or intended for synchronous systems [11,24], which changes when PBFT is published. PBFT is a replication algorithm to tolerate Byzantine faults. PBFT or its variants are used in many existing permissioned blockchain systems, such as Hyperledger [5], Parity [3], Ripple [6] and ByzCoin [13]. However, the blockchain based on classic PBFT protocol keeps the fault tolerance rate of $\frac{f}{3f+1}$—which is low, the communication bound of $O(N^2)$ where N is the number of nodes. Thus, the performance of blockchain system based on PBFT algorithm is neither effective nor efficient at present, which implies the PBFT based protocols hurt scaling. Comparing to PBFT, XFT [20] can handle Byzantine, crash and network faults together whose fault tolerance rate is $\frac{f}{2f+1}$—higher than PBFT. In addition, it is more efficient than PBFT [20].

PoW (Proof of work) is a consensus strategy used in the Bitcoin network [22]. In PoW, each node of the network calculates a hash value of the block header. Generally the work means huge calculations. Thus it consumes much electricity and provides a very low TPS. To get a higher TPS, Conflux [19] represents relationships between blocks as a direct acyclic graph and achieves consensus on a total order of the blocks. But it sacrifice a lot of memories to save redundant transactions.

PoS (Proof of stake) is another energy-saving alternative to PoW [12]. Popular blockchain systems such as Ethereum [4] adopts PoS as their consensus algorithm. Miners in PoS have to prove the ownership of the amount of currency instead of the computation consumption. However, the selection based on account balance is quite unfair because the single richest person is bound to be dominant in the network. As a result, many solutions are proposed to improve PoS. For example, Blackcoin [28] uses randomization to predict the next generator and Peercoin [12] favors coin age based selection. However, both of them are not thorough solutions to the problem.

The major difference between PoS and DPoS [1] is that PoS is direct democratic while DPoS is representative democratic. In DPoS, stakeholders elect their

delegates to generate and validate blocks. With significantly fewer nodes to validate blocks, the block could be confirmed quickly, leading to the quick confirmation of transactions. However, for DPoS, there are many difficulties in dealing with faulty nodes. The validator elections cannot effectively prevent the emergence of some destructive nodes and cause potential safety hazard to the network.

5 Conclusion and Future Work

In this paper, we propose Crux, a novel algorithm to achieve flexible, fast consensus with high fault tolerance rate in blockchain. Our algorithm first employs Delegated Proof of Stake to elect $2f + 1$ validators in every epoch to generate blocks and confirm them. Each epoch is composed of a series of views. In each view $f + 1$ validators are determined to be active which compromise the synchronous group. Among them, one is the primary that collect transactions and generate blocks and the others are followers which check the validity of the generated blocks through message deliveries. If invalid behaviors are detected, current active validators invoke the process of view change to alter the members of synchronous group in the new view until there is no incorrect or partitioned validator exists in the group.

There is a main disadvantage in our Crux now—when the number of incorrect or partitioned validators is large (*e.g.*, near f), the time consumption of view change will be huge. Therefore, a fault detection mechanism will be one of our future works. Fortunately, it may not be very difficult. An active validator in sg_{i+1} could detect which validator behaves invalidly based on the $CommitLog_*$ and $PrepareLog_*$ in the VIEW-CHANGE messages—this inspires us to make the VIEW-CHANGE messages contain both of commit and prepare logs. Moreover, we can also design an approach to directly select correct and synchronous validators in the process of view change.

References

1. Delegated proof-of-stake consensus. https://bitshares.org/technology/delegated-proof-of-stake-consensus/
2. EOSIO white paper. https://github.com/EOSIO/Documentation/blob/master/TechnicalWhitePapermd
3. Ethcore. parity: next generation ethereum browser. https://ethcore.io/parity.html
4. Ethereum blockchain app platform. https://www.ethereum.org
5. Hyperledger project. https://www.hyperledger.org/projects/fabric
6. Ripple. https://ripple.com
7. Bellare, M., Rogaway, P.: The exact security of digital signatures-how to sign with RSA and rabin. In: Maurer, U. (ed.) EUROCRYPT 1996. LNCS, vol. 1070, pp. 399–416. Springer, Heidelberg (1996). https://doi.org/10.1007/3-540-68339-9_34
8. Castro, M., Liskov, B.: Practical byzantine fault tolerance. In: OSDI, pp. 173–186 (1999)
9. Castro, M., Liskov, B.: Practical byzantine fault tolerance and proactive recovery. ACM Trans. Comput. Syst. **20**(4), 398–461 (2002)

10. Clement, A., Wong, E.L., Alvisi, L., Dahlin, M., Marchetti, M.: Making byzantine fault tolerant systems tolerate byzantine faults. In: NSDI, pp. 153–168 (2009)
11. Hopkins, A.L., Lala, J.H., Smith, T.B.: The evolution of fault tolerant computing at the Charles Stark Draper laboratory, 1955–85. In: Avižienis, A., Kopetz, H., Laprie, J.C. (eds.) The Evolution of Fault-Tolerant Computing. DEPENDABLECOMP, vol. 1, pp. 121–140. Springer, Vienna (1987). https://doi.org/10.1007/978-3-7091-8871-2_6
12. King, S., Nadal, S.: PPcoin: peer-to-peer crypto-currency with proof-of-stake. Self-published Paper, 19 August 2012
13. Kogias, E.K., Jovanovic, P., Gailly, N., Khoffi, I., Gasser, L., Ford, B.: Enhancing bitcoin security and performance with strong consistency via collective signing. In: USENIX Security, pp. 279–296 (2016)
14. Kotla, R., Alvisi, L., Dahlin, M., Clement, A., Wong, E.L.: Zyzzyva: speculative byzantine fault tolerance. ACM Trans. Comput. Syst. **27**(4), 7:1–7:39 (2009)
15. Kotla, R., Alvisi, L., Dahlin, M., Clement, A., Wong, E.: Zyzzyva: speculative byzantine fault tolerance. ACM SIGOPS Oper. Syst. Rev. **41**(6), 45–58 (2007)
16. Lamport, L., Shostak, R.E., Pease, M.C.: The byzantine generals problem. ACM Trans. Program. Lang. Syst. **4**(3), 382–401 (1982)
17. Lamport, L., et al.: Paxos made simple. ACM SIGACT News **32**(4), 18–25 (2001)
18. Larimer, D.: Delegated proof-of-stake (DPOS). Bitshare whitepaper (2014)
19. Li, C., Li, P., Xu, W., Long, F., Yao, A.C.: Scaling Nakamoto consensus to thousands of transactions per second. CoRR abs/1805.03870 (2018)
20. Liu, S., Viotti, P., Cachin, C., Quéma, V., Vukolic, M.: XFT: practical fault tolerance beyond crashes. In: OSDI, pp. 485–500 (2016)
21. McConaghy, T., et al.: BigchainDB: a scalable blockchain database. BigchainDB white paper (2016)
22. Nakamoto, S.: Bitcoin: a peer-to-peer electronic cash system (2008)
23. Ongaro, D., Ousterhout, J.K.: In search of an understandable consensus algorithm. In: USENIX ATC, pp. 305–319 (2014)
24. Paulitsch, M., Morris, J., Hall, B., Driscoll, K., Latronico, E., Koopman, P.: Coverage and the use of cyclic redundancy codes in ultra-dependable systems. In: DSN, pp. 346–355 (2005)
25. Rivest, R.L.: The MD5 message-digest algorithm. RFC 1321, pp. 1–21 (1992)
26. Rivest, R.L., Shamir, A., Adleman, L.M.: A method for obtaining digital signatures and public-key cryptosystems. Commun. ACM **21**(2), 120–126 (1978)
27. Tsudik, G.: Message authentication with one-way hash functions. In: INFOCOM, pp. 2055–2059 (1992)
28. Vasin, P.: Blackcoin's proof-of-stake protocol v2 (2014). https://blackcoin.co/blackcoin-pos-protocolv2-whitepaper.pdf